Bst Traversal

AlgoExpert · Medium · Binary Search Trees

The problem

- In-order, pre-order and post-order traversals appending into a list.

Intuition

- In-order, pre-order and post-order differ only in when the node is visited relative to its children.

Approach

- 1. In-order: left, node, right
- 2. Pre-order: node, left, right
- 3. Post-order: left, right, node

Java solution

```java
public class BstTraversal {
    public static List<Integer> inOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { inOrderTraverse(t.left, out); out.add(t.value); inOrderTraverse(t.right, out); }
        return out;
    }

    public static List<Integer> preOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { out.add(t.value); preOrderTraverse(t.left, out); preOrderTraverse(t.right, out); }
        return out;
    }

    public static List<Integer> postOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { postOrderTraverse(t.left, out); postOrderTraverse(t.right, out); out.add(t.value); }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Recap

- In-order, pre-order and post-order differ only in when the node is visited relative to its children.
- Complexity: O(n)